

智能联网时钟系统

嵌入式系统设计大作业简介

大作业名称： 智能联网时钟系统

学生姓名： 江彦佐

学 号： 524442910013

专 业： 自动化

班 级： 自感 2421

完成日期： 2026 年 6 月 15 日

本报告为嵌入式系统设计课程大作业提交材料

目录

1	系统概述与整体架构	3
1.1	系统简介	3
1.2	系统架构框图	3
1.3	数据流转逻辑	4
2	协议规范与容错设计	4
2.1	通信参数	4
2.2	容错三件套	4
2.2.1	大小写不敏感	4
2.2.2	空格与制表符多重容错	5
2.2.3	指令大写缩写识别	5
2.3	FORMAT RIGHT 逆序传输机制	6
3	关键代码片段与实现细节	7
3.1	PC 端串口后台线程 (serial_worker.py)	7
3.1.1	设计目标	7
3.1.2	信号接口	7
3.1.3	线程主循环	7
3.2	PC 端数字孪生镜像面板 (twin_panel.py)	8
3.2.1	自绘 7SEG 数码管控件 (SevenSegmentDigit)	8
3.2.2	双向同步逻辑	9
3.2.3	长按检测机制	10
3.2.4	LED 控件 (LedIndicator)	11
3.2.5	接收更新入口 (main.py)	11
3.3	MCU 端中断系统与实时调度	12
3.3.1	设计思想	12
3.3.2	SysTick 系统时基	12
3.3.3	Timer0A 显示扫描与按键采样	13
3.3.4	UART 中断驱动收发	14
3.4	MCU 端数码管动态扫描	15
3.4.1	显示刷新 (Display_Refresh)	15
3.4.2	显示内容构建 (Build_Display)	16
3.4.3	段码转换 (SegCode)	16
3.5	MCU 端按键状态机与事件分发	16
3.5.1	按键扫描 (Key_Scan)	16

3.5.2	事件分发 (Dispatch_Key)	17
4	扩展功能实现说明	18
4.1	E1: NTP 网络异步对时	18
4.1.1	动机	18
4.1.2	实现	18
4.2	E2: 天气获取与板端联动	19
4.2.1	动机	19
4.2.2	实现	19
4.3	E3: 自动昼夜模式	20
4.3.1	实现	20
4.4	E4: 数据可视化看板	21
4.4.1	数据持久化	21
4.4.2	三张并列图表 (chart_widget.py)	21
5	自主增加功能: 专注倒计时系统	22
5.1	动机	22
5.2	设计	22
5.3	实现	22
5.4	演示	24
6	难点解决与防崩溃机制	24
6.1	难点一: 高频心跳包刷屏导致日志卡顿	24
6.2	难点二: 网络超时导致 UI 假死	25
6.3	难点三: 极端工程边界下的防崩溃设计	25
7	相关文件	27

1 系统概述与整体架构

1.1 系统简介

本系统基于 **S800 硬件板卡 (TI TM4C1294NCPDT)** 作为下位机, **Python 3.11 + PyQt5** 作为 PC 上位机, 通过 **USB 虚拟串口** 实现 **1:1 数字孪生镜像的双向协同**。

- **硬件外设层**

组成: S800 板卡 + I²C 7SEG 数码管 + I²C 按键 + GPIO USER1/USER2 + PCA9557 LED + 蜂鸣器 PK5

职责: 完成时钟走时、显示刷新、按键检测、闹钟触发等实时任务。

- **串口协议层**

组成: UART0 中断驱动收发 + ring buffer 行拼装 + Token_Matches() 容错解析

职责: 实现命令容错识别、多参数组合提取以及 FORMAT RIGHT 逆序响应功能。

- **PC 后台通信层**

组成: SerialWorker (QThread) + NetworkWorker (QThread)

职责: 负责串口收发以及 NTP、天气数据的异步请求, 跨线程通知 UI 更新。

- **PC UI 渲染层**

组成: PyQt5 MainWindow + TwinPanel + QPlainTextEdit 日志

职责: 完成数字孪生渲染、控制面板交互、四色日志显示以及数据看板展示。

1.2 系统架构框图

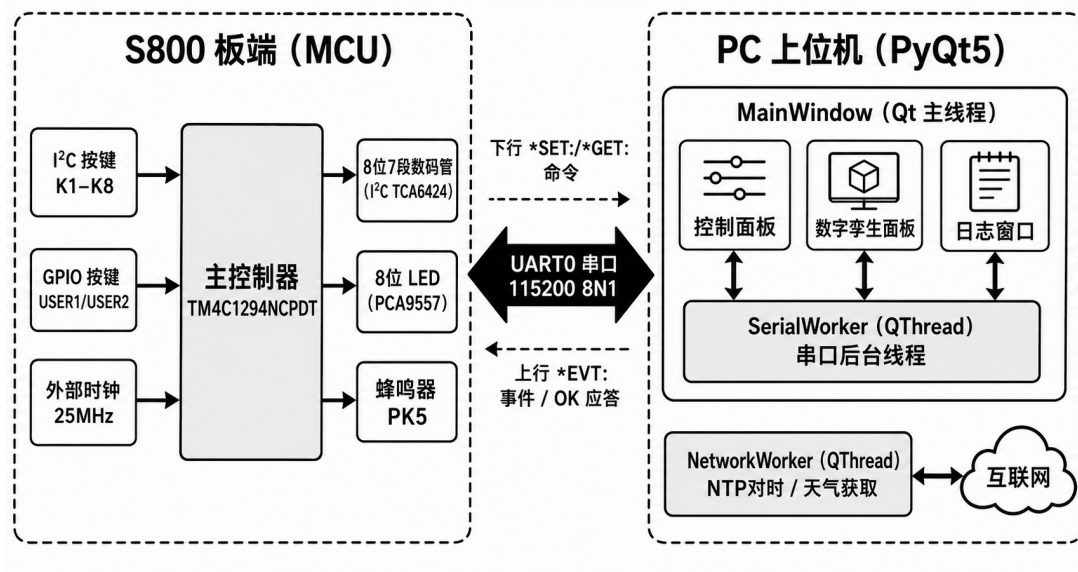


图 1: 系统整体架构框图

1.3 数据流转逻辑

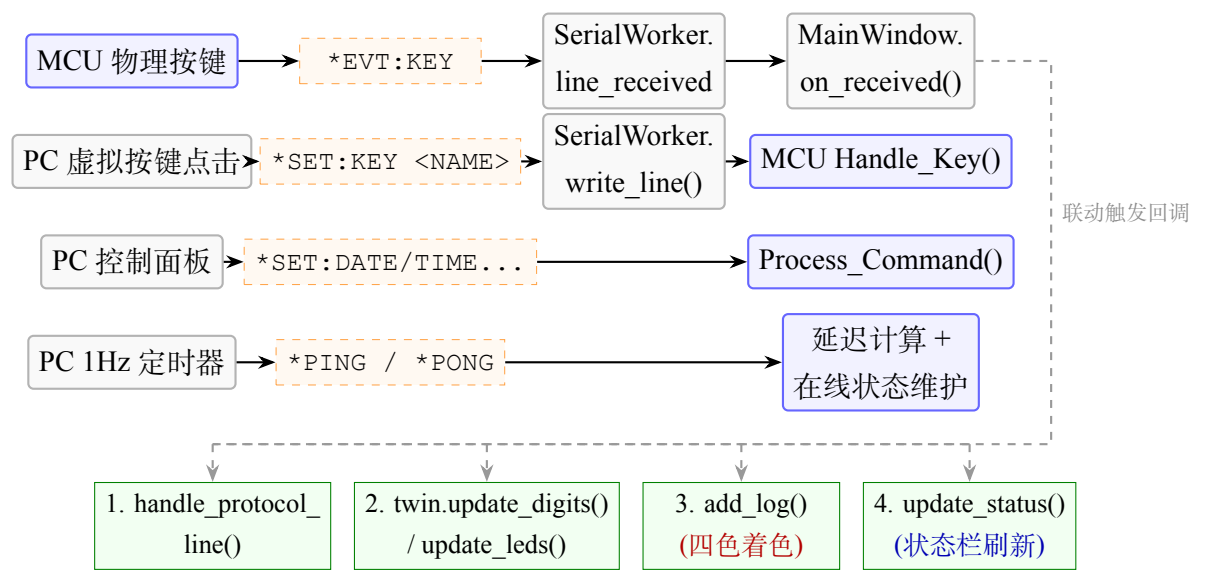


图 2: 系统数据流转与通信协议逻辑示意图

2 协议规范与容错设计

2.1 通信参数

表 1: 串口通信参数

参数	规格
物理层	USB 虚拟串口
波特率	115200
数据格式	8 数据位，无校验，1 停止位，无流控
字符编码	ASCII
行结束符	\r\n 或 \n 兼容
单帧最大长度	64 字节

2.2 容错三件套

2.2.1 大小写不敏感

MCU 端 Process_Command() 收到一行指令后，首先调用 Upper_Copy() (main.c:1605-1610) 将整行处理为大写：

```

1 static void Upper_Copy(char *dst, const char *src, uint16_t max)
2 {
3     uint16_t i;
4     for (i = 0; i < max - 1 && src[i]; i++)
5         dst[i] = (char)toupper((unsigned char)src[i]);
6     dst[i] = '\0';
7 }

```

因此 *SET:TIME 与 *set:time 完全等价。

2.2.2 空格与制表符多重容错

命令行通过 strtok(work, " \t") 分割参数 (main.c:1674), 任意数量的空格或 Tab 均可正确分割:

```

1 tok = strtok(work, " \t");
2 while (tok && argc < 14) {
3     argv[argc++] = tok;
4     tok = strtok(NULL, " \t");
5 }

```

如 *SET: TIME HOUR 12 与 *SET:TIME HOUR 12 同样合法。

2.2.3 指令大写缩写识别

Token_Matches() (main.c:1616-1627) 是实现缩写识别的关键函数:

```

1 static bool Token_Matches(const char *token, const char *pattern)
2 {
3     char t[24], p[24];
4     uint8_t req = 0, i;
5     Upper_Copy(t, token, sizeof(t));
6     Upper_Copy(p, pattern, sizeof(p));
7     // 统计 pattern 中大写字母数量 即最少必输字符数
8     for (i = 0; pattern[i]; i++)
9         if (isupper((unsigned char)pattern[i])) req++;
10    // token 长度必须 >= 必输字符数 <= 完整 pattern 长度
11    if (strlen(t) < req || strlen(t) > strlen(p)) return false;
12    return strncmp(p, t, strlen(t)) == 0;
13 }

```

原理：模式串 MINute 中，大写字母为 M、I、N，小写字母 ute 可省略。用户输入至少 3 个字符且为模式串前缀即视为合法：

- MIN → 3 字符 ≥ req=3，前缀匹配 → **合法**
- MINU → 4 字符 ≥ req=3，前缀匹配 → **合法**
- MI → 2 字符 < req=3 → **拒绝**

当 pattern 全部为大写字母时(如 ON、OFF、LEFT), req = strlen(pattern), 即必须完整输入，防止误匹配。

2.3 FORMAT RIGHT 逆序传输机制

发送 *SET:FORMAT RIGHT 后,MCU 将 g_format 切换为 FMT_RIGHT(main.c:1771)。此后所有 *GET 应答和 *EVT:DISP 上报均触发逆序逻辑。

Send_OK_Value() (main.c:1650-1662):

```
1 static void Send_OK_Value(const char *value)
2 {
3     char out[32];
4     uint8_t len = (uint8_t)strlen(value), i;
5     if (g_format == FMT_LEFT) {
6         UART_Printf("OK %s\r\n", value);
7         return;
8     }
9     // RIGHT 模式下逐字符逆序
10    for (i = 0; i < len; i++) out[i] = value[len - 1U - i];
11    out[len] = '\0';
12    UART_Printf("OK %s\r\n", out);
13 }
```

示例：当前时间 12:30:45，PC 查询：

表 2: FORMAT LEFT/RIGHT 应答对比

项目	LEFT	RIGHT
*GET:TIME 应答	OK 12.30.45	OK 54.03.21
数码管显示	12.30.45	54.03.21

数码管显示同步逆序，小数点位置按下一位规则跟随，逆序后 dp 提前一位。

PC 端协议层解码 (protocol.py):

```
1 def parse_disp_event(line: str):
```

```

2     """解析 *EVT:DISP <8字符> <dpHex>, dpHex 为 2 位 hex 表示 8 位小
    数点"""
3     body = line[len("*EVT:DISP "):]
4     payload = body[:-3][:8].ljust(8).replace("_", " ")
5     dp = int(body[-2:], 16)
6     return payload, dp
7
8 def parse_led_event(line: str) -> int:
9     return int(line.split()[-1], 16)

```

PC 端 FORMAT 状态同步 (main.py:585-603): PC 收到 OK LEFT/OK RIGHT 后更新 state.fmt, 同时兼容 RIGHT 模式下 MCU 上报的逆序字符串 TFEL/THGIR。

3 关键代码片段与实现细节

3.1 PC 端串口后台线程 (serial_worker.py)

3.1.1 设计目标

将串口 I/O 从 Qt 主线程中完全剥离, 通过 pyqtSignal 跨线程通信, 确保 1Hz 心跳、*EVT:DISP 事件高频上报时 GUI 永不卡顿。

3.1.2 信号接口

```

1 class SerialWorker(QThread):
2     line_received = pyqtSignal(str)           # 收到一行完整报文 -> 主线程协议解析
3     connection_changed = pyqtSignal(bool)     # 连接状态变化 -> 更新 UI 指示灯
4     latency_updated = pyqtSignal(int)         # 心跳延迟更新 (ms)
5     error = pyqtSignal(str)                   # 错误信息 -> 弹窗 + 红色日志 + 禁用控件

```

3.1.3 线程主循环

```

1 def run(self):
2     self._running = True
3     try:
4         self._ser = serial.Serial(

```

```

5         self.port, self.baud,
6         timeout=0.1,          # 100ms 读超时, 避免阻塞卡死
7         write_timeout=0.4,
8         dsrdtr=False, rtscts=False, xonxoff=False,
9     )
10    self.connection_changed.emit(True)
11    except Exception as exc:
12        self.error.emit(f"open {self.port} failed: {exc}")
13    return
14
15    buf = bytearray() # ring buffer 拼行
16    while self._running:
17        try:
18            while not self._tx.empty(): # 消费写队列
19                self._ser.write(
20                    self._tx.get_nowait().encode("ascii", errors=
21                    "ignore"))
22            chunk = self._ser.read(128) # 非阻塞读 128 字节
23            if chunk:
24                buf.extend(chunk)
25                while b"\n" in buf: # 按 \n 分割完整行
26                    line, _, rest = buf.partition(b"\n")
27                    buf = bytearray(rest)
28                    text = line.decode("ascii", errors="replace")
29                    .strip()
30                    if text:
31                        self.line_received.emit(text) # 跨线程发送
32                    else:
33                        time.sleep(0.005) # 无数据时让出 CPU
34        except Exception as exc:
35            self.error.emit(f"serial error: {exc}")
36            break

```

3.2 PC 端数字孪生镜像面板 (twin_panel.py)

3.2.1 自绘 7SEG 数码管控件 (SevenSegmentDigit)

使用 QPainter 在 paintEvent 中逐段绘制:

```
1 def paintEvent(self, _event):
2     p = QPainter(self)
3     p.setRenderHint(QPainter.Antialiasing)
4     w, h = self.width(), self.height()
5     active = QColor("#FF3030")      # 亮段: LED 红
6     inactive = QColor("#220000")    # 暗段: 暗红底色
7     p.fillRect(self.rect(), QColor("#050505")) # 深黑背景
8
9     on = "" if (self.blink and not self._blink_visible) \
10            else self.MAP.get(self.char, "")
11     for name, rect in self.SEGMENTS.items(): # 7 段
12         x, y, rw, rh = rect
13         p.setBrush(QBrush(active if name in on else inactive))
14         p.setPen(Qt.NoPen)
15         p.drawRoundedRect(QRectF(x * w, y * h, rw * w, rh * h),
16                            3, 3)
17         # 小数点
18         p.setBrush(QBrush(active if self.dp else inactive))
19         p.drawEllipse(QRectF(0.82 * w, 0.87 * h, 0.12 * w, 0.08 * h))
```

表 3: 7SEG 段坐标 (相对坐标 0.0–1.0)

段	位置 (x, y, w, h)	说明
a	(0.22, 0.06, 0.56, 0.08)	上横
b	(0.78, 0.13, 0.08, 0.33)	右上竖
c	(0.78, 0.54, 0.08, 0.33)	右下竖
d	(0.22, 0.87, 0.56, 0.08)	下横
e	(0.14, 0.54, 0.08, 0.33)	左下竖
f	(0.14, 0.13, 0.08, 0.33)	左上竖
g	(0.22, 0.47, 0.56, 0.08)	中横
dp	(0.82, 0.87, 0.12, 0.08)	小数点

字符映射表覆盖 0–9、A–Z、-、_、空格共 38 个字符，段码与 MCU 端保持一致。

3.2.2 双向同步逻辑

下行 (PC→MCU): 虚拟按键 → *SET:KEY

```
1 def on_virtual_key(self, name: str):
```

```

2     if name == "USER1":           # USER1 短按 = 请求 PC 对时
3         self.ntp_sync()
4         return
5     self.send_command(f"*SET:KEY {name}")
6
7 def on_virtual_long_key(self, name: str):
8     if name == "FUNC":           # FUNC 长按 = SAVE
9         self.send_command("*SET:KEY SAVE")
10    elif name == "ADD":           # ADD 长按 = 连加 3 次 (200ms 间隔)
11        for i in range(3):
12            QTimer.singleShot(i * 200,
13                               lambda: self.send_command("*SET:KEY ADD"))
14    elif name == "USER1":         # USER1 长按 = 板端 NTP 状态短显
15        self.send_command("*SET:KEY USER1")

```

上行 (MCU→PC): *EVT:KEY → 按钮高亮 200ms

```

1 def pulse_key(self, name: str):
2     key = normalize_key_name(name)
3     btn = self.buttons.get(key)
4     if not btn:
5         return
6     btn.setProperty("pulse", True)
7     btn.style().unpolish(btn)
8     btn.style().polish(btn)
9     QTimer.singleShot(200, lambda b=btn: self._clear_pulse(b))

```

3.2.3 长按检测机制

pressed 信号触发后启动定时器, released 在定时器到期前发生则视为短按:

```

1 def _on_pressed(self, name: str):
2     self._pressed[name] = True
3     self._long_sent[name] = False
4     QTimer.singleShot(800, lambda n=name: self._emit_long(n))
5
6 def _on_released(self, name: str):
7     if self._pressed.get(name) \
8         and not self._long_sent.get(name):
9         self.key_clicked.emit(name)    # 短按

```

```

10     self._pressed[name] = False
11
12     def _emit_long(self, name: str):
13         if self._pressed.get(name):
14             self._long_sent[name] = True
15             self.key_long.emit(name)          # 长按

```

3.2.4 LED 控件 (LedIndicator)

LED 采用 QSS `radialgradient` 径向渐变绘制, 切换 `objectName` 实现亮灭:

```

1 class LedIndicator(QWidget):
2     def __init__(self, index: int, hint: str):
3         super().__init__()
4         self._dot = QLabel()
5         self._dot.setObjectName("led_off")
6         self._dot.setFixedSize(30, 30)
7         caption = QLabel(f"L{index + 1}\n{hint}")
8
9         def set_on(self, on: bool):
10             self._dot.setObjectName("led_on" if on else "led_off")
11             self._dot.style().unpolish(self._dot)
12             self._dot.style().polish(self._dot)

```

亮态 QSS: `radialgradient(cx:0.5, cy:0.5, radius:0.5, stop:0 #FFFF80, stop:1 #FFC800)`。

3.2.5 接收更新入口 (main.py)

```

1 def handle_protocol_line(self, line: str):
2     if line.startswith("*PONG"):
3         self.last_pong_time = time.time()
4         self.state.online = True
5         latency = int((time.time() - self.last_ping_time) * 1000)
6         self.worker.latency_updated.emit(latency)
7         return
8     if line.startswith("*EVT:DISP"):
9         text, dp = parse_disp_event(line)
10        self.twin.update_digits(text, dp)
11        return

```

```
12     if line.startswith("*EVT:LED"):
13         self.twin.update_leds(parse_led_event(line))
14         return
15     if line.startswith("*EVT:KEY"):
16         name = line.split()[-1]
17         self.twin.pulse_key(name)
18         if name == "USER1":
19             self.ntp_sync()
20         return
21     if line.startswith("*EVT:ALARM"):
22         QMessageBox.information(self, "闹钟", "S800 闹钟正在响铃。")
23         return
24     if line.startswith("*EVT:MODE"):
25         self.state.mode = "NIGHT" if "NIGHT" in line else "DAY"
26         self.update_status()
27         return
```

3.3 MCU 端中断系统与实时调度

3.3.1 设计思想

MCU 端采用中断优先、前后台协作的架构：所有硬实时任务，如显示扫描、按键采样、UART 收发等在中断服务例程中完成，主循环仅负责非实时的业务逻辑如命令解析、显示内容计算、状态机推进等。这种设计使得动态扫描不会卡顿，串口 FIFO 不会溢出。

3.3.2 SysTick 系统时基

SysTick_Handler() 每1毫秒触发一次,驱动四个分频标志(main.c:253-259):

```
1 void SysTick_Handler(void)
2 {
3     g_tick_ms++;
4     if ((g_tick_ms % 5U) == 0U)    flag_5ms = 1;
5     if ((g_tick_ms % 10U) == 0U)   flag_10ms = 1;
6     if ((g_tick_ms % 100U) == 0U)  flag_100ms = 1;
7     if ((g_tick_ms % 1000U) == 0U) flag_1s = 1;
8 }
```

全局变量 g_tick_ms 是全部定时和超时的单一基准时钟源。所有计时函数均通过 Tick_TimedOut(start, span) 计算差值, 规避 int 溢出的风险。

3.3.3 Timer0A 显示扫描与按键采样

TIMER0A_Handler() 每 1 毫秒执行一次 (main.c:270-293), 是系统最关键的中断服务例程:

```
1 void TIMER0A_Handler(void)
2 {
3     static uint8_t key_div = 0;
4
5     TimerIntClear(TIMER0_BASE, TIMER_TIMA_TIMEOUT);
6
7     Display_Refresh();
8
9     if (g_led_dirty) {
10         g_led_dirty = false;
11         I2C0_WriteByte(PCA9557_I2CADDR, PCA9557_OUTPUT,
12                        (uint8_t)~g_led_byte);
13     }
14
15     if (++key_div >= KEY_SCAN_TICKS) {
16         uint8_t tca = I2C0_ReadByte(TCA6424_I2CADDR,
17                                    TCA6424_INPUT_PORT0);
18
19         uint16_t raw = 0;
20         uint8_t i;
21         key_div = 0;
22         for (i = 0; i < 8; i++)
23             if ((tca & (1U << i)) == 0) raw |= (1U << i);
24         g_tca_key_raw = (uint8_t)raw;
25     }
```

关键设计要点:

- 所有 I²C 总线访问都集中在此 ISR 内, 从而消除总线竞争
- LED 更新采用标志 g_led_dirty 延迟写入, 避免每次逻辑变更都触发阻塞写入
- 按键采样以 20 毫秒间隔分频, 采样结果供主循环中的 Key_Scan() 使用
- I²C 异常检测与恢复: 连续 3 次写入失败时设置 g_i2c_recover_request, 由主循环调用 I2C0_BusRecover() 恢复, 避免在 ISR 中执行耗时的总线恢复操作

3.3.4 UART 中断驱动收发

UART 接收采用最高优先级中断 (main.c:300-320)，接收中断优先级最高，确保在 I²C 事务进行时到达的字节不会因 FIFO 溢出而丢失。ISR 仅操作 ring buffer，绝不访问 I²C，因此抢占显示扫描 ISR 是安全的。

UART_RxIsrHandler() (main.c:325-362) 的逐行组装逻辑：

```

1  static void UART_RxIsrHandler(uint8_t byte)
2  {
3      uint8_t i;
4
5      if (byte == '\r' || byte == '\n') {
6          if (g_rx_asm_ovf) {
7              g_rx_asm_ovf = false;
8              g_rx_asm_len = 0;
9              g_line_too_long = true;
10             return;
11         }
12         if (g_rx_asm_len == 0) return;    // 忽略空行
13         Mark_Serial_Activity();
14         for (i = 0; i < g_rx_asm_len; i++) {
15             uint16_t next = (uint16_t)((g_rx_head + 1U) %
RX_RING_SIZE);
16             if (next != g_rx_tail) {
17                 g_rx_ring[g_rx_head] = g_rx_asm[i];
18                 g_rx_head = next;
19             }
20         }
21         {
22             uint16_t next = (uint16_t)((g_rx_head + 1U) %
RX_RING_SIZE);
23             if (next != g_rx_tail) {
24                 g_rx_ring[g_rx_head] = '\n';
25                 g_rx_head = next;
26             }
27         }
28         rx_line_ready++;
29         g_rx_asm_len = 0;
30         return;
31     }

```

```

32
33     if (g_rx_asm_len >= LINE_MAX) {
34         g_rx_asm_ovf = true;    // 继续消耗字节直到 CR/LF
35         return;
36     }
37     g_rx_asm[g_rx_asm_len++] = (char)byte;
38 }

```

设计决策：采用两级缓冲——`g_rx_asm[ ]`（行组装缓冲）→`g_rx_ring[ ]`（环形队列）。行结束符到达时整行原子提交到 `ring`，主循环通过 `UART_GetLine()` 从中提取完整命令行，调用 `Process_Command()` 处理。

3.4 MCU 端数码管动态扫描

3.4.1 显示刷新 (Display_Refresh)

`Display_Refresh()` (`main.c:851-898`) 每秒被 `Timer0A ISR` 调用 1000 次，每次刷新一位数码管（共 8 位），形成稳定的 125 Hz 刷新率（ $1000/8$ ）。

```

1  static void Display_Refresh(void)
2  {
3      static bool blanked = false;
4      uint8_t code;
5      uint8_t err = 0;
6      bool blink_off;
7
8      if (!disp_on) {
9          /* 仅在首次进入熄灭态时写一次 I2C 若每个 tick 都写两次字节
10             会在高频 ISR 中卡住足够长时间饿死 UART RX FIFO */
11         if (!blanked) {
12             I2C0_WriteByte(..., OUTPUT_PORT2, 0x00);
13             I2C0_WriteByte(..., OUTPUT_PORT1, 0x00);
14             blanked = true;
15         }
16         return;
17     }
18     blanked = false;
19
20     if (g_scan_pos >= g_scan_limit)
21         g_scan_pos = 0;

```

```
22
23     blink_off = (disp_blink_mask & (1U << g_scan_pos))
24                 && !g_blink_on;
25     code = blink_off ? 0U : SegCode(disp_buf[g_scan_pos]);
26     if (!blink_off && disp_dp[g_scan_pos]) code |= 0x80;
27
28     // 先清零位选消除鬼影 再通过一次自动递增突发写入段码和新位选
29     I2C0_WriteByte(..., OUTPUT_PORT2, 0x00);
30     I2C0_WriteTwo(..., AUTO_INC | OUTPUT_PORT1,
31                   code, (1U << g_scan_pos));
32 }
```

防鬼影策略：每次位切换时先向 PORT2 写入 0x00（消隐），再写入新段码和新位选。端口 1（段码）和端口 2（位选）使用 TCA6424 的自动递增特性（AUTO_INC），在单次 I²C burst 中完成，杜绝残影。

3.4.2 显示内容构建 (Build_Display)

主循环每秒调用一次 Build_Display() (main.c:985) ，按优先级组装 disp_buf[8]：

1. **倒计时优先：**若 g_countdown_active 为真，显示格式 CDMSS，第 4 位小数点常亮
2. **天气短显：**若 g_weather_until_ms 未到期，显示天气文本，过期数据闪烁提示
3. **流水消息：**若 g_message_until_ms 未到期，执行流水滚动，Scroll_Tick() 负责方向与速度控制
4. **正常时钟：**根据当前显示模式（时间 / 日期 / 年份）格式化，NIGHT 模式下仅显示前 4 位（时 + 分）

3.4.3 段码转换 (SegCode)

SegCode() (main.c:812-846) 将 ASCII 字符映射为 8 位段码 (bit 0–6 对应 a–g, bit 7 为小数点)，覆盖数字 0–9、大写字母 A–Z、横线 - 和下划线 _。段码表与 PC 端 SevenSegmentDigit.MAP 保持一致，确保数字孪生镜像的字符渲染完全一致。

3.5 MCU 端按键状态机与事件分发

3.5.1 按键扫描 (Key_Scan)

Key_Scan() (main.c:1159) 在 flag_10ms 触发时由主循环调用，完成：

1. **数据融合：**合并 I²C 按键和 GPIO 按键 (USER1/2, PJ0/PJ1)

2. **软件去抖**: 连续 KEY_DEBOUNCE_COUNT 次采样结果一致才认为状态稳定
3. **边缘检测**: 稳定值变化时识别按下和释放事件, 推入事件队列
4. **长按与连发**: 按下后基于 g_tick_ms 计时, 超过 800 毫秒产生 KEV_LONG 事件; ADD 键在编辑态下每 200 毫秒产生 KEV_REPEAT

```

1 // 长按与连发由时间驱动 与边沿检测独立
2 if (g_key_fsm[b] == 1 &&
3     Tick_TimedOut(g_key_press_ms[b], FUNC_LONG_MS)) {
4     g_key_fsm[b] = 2;
5     Key_Push(code, KEV_LONG);
6 }
7 if (code == KEY_ADD && g_edit_state != ST_IDLE) {
8     if (Tick_TimedOut(g_key_repeat_ms[b], ADD_REPEAT_MS)) {
9         g_key_repeat_ms[b] = g_tick_ms;
10        Key_Push(code, KEV_REPEAT);
11    }
12 }

```

3.5.2 事件分发 (Dispatch_Key)

Dispatch_Key()(main.c:1222)将事件类型分发到具体行为函数 Handle_Key() 或直接处理长按/连发:

- FUNC 键: 短按在编辑态循环切换模式 (日期 → 时间 → 闹钟); 长按等效 SAVE (保存退出); 响铃中短按优先关闹钟
- SHIFT 键: 编辑态下循环切换高亮字段
- ADD 键: 当前编辑字段循环加 1, 含进位与范围钳制
- SAVE 键: 保存编辑缓冲区的值并退出编辑态
- DISP / SPEED / FORMAT / EXT: 切换显示模式、流水速度、流水方向、触发 EXT 事件上报
- USER1: 上报 *EVT:KEY USER1 请求 PC 对时
- USER2: 若天气数据有效则短显 5 秒, 否则显示 --C---

按键事件通过长度为 8 的环形队列 (g_evq[1]) 解耦 ISR 与主循环, ISR 仅推入事件, 主循环取出, 避免阻塞。

4 扩展功能实现说明

4.1 E1: NTP 网络异步对时

4.1.1 动机

板载 MCU 无后备电池供电的 RTC，每次上电需从 PC 获取标准网络时间校准。

4.1.2 实现

ntp_helper.py —— 三服务器容错策略：

```
1  SERVERS = ("ntp.aliyun.com", "cn.ntp.org.cn", "ntp.ntsc.ac.cn")
2
3  def fetch_network_time(timeout=3):
4      last_error = None
5      for server in SERVERS:
6          try:
7              response = ntplib.NTPClient().request(
8                  server, version=3, timeout=timeout)
9              network_time = dt.datetime.fromtimestamp(response.
10 tx_time)
11              delta_ms = int((network_time - dt.datetime.now())
12                             .total_seconds() * 1000)
13              return network_time, delta_ms, server
14          except Exception as exc:
15              last_error = exc
16      raise RuntimeError(f"NTP unavailable: {last_error}")
```

main.py —— 异步调度：

```
1  def ntp_sync(self):
2      self.net_worker = NetworkWorker("ntp")
3      self.net_worker.done.connect(self.on_network_done)
4      self.net_worker.failed.connect(self.on_network_failed)
5      self.net_worker.start()
6
7  def on_network_done(self, kind, data):
8      now, delta_ms, server = data
9      self.send_command(
10         f"*SET:DATE YEAR MONTH DATE {now.year:04d} {now.month:02d}
```

```
        } {now.day:02d}")
11     self.send_command(
12         f"*SET:TIME HOUR MIN SEC {now.hour:02d} {now.minute:02d}
        {now.second:02d}")
13     self.send_command("*NTP SYNC")
14     self.store.append("SYNC", f"delta {delta_ms}")
15     self.add_log("SYS", f"NTP sync {server} delta {delta_ms} ms")
```

表 4: NTP 对时触发方式

方式	路径
PC 点击 [NTP 对时] 按钮	ntp_sync() 直接调用
MCU 按下 USER1	*EVT:KEY USER1 → PC 自动触发 ntp_sync()
网络失败	弹窗 + 红色日志告警

LED 指示: MCU 收到 *NTP SYNC 后置位 g_ntp_synced, LED8 (NTP) 常亮表示已同步; 超过 24 小时未同步时 LED8 慢闪 (DRIFT 状态)。

4.2 E2: 天气获取与板端联动

4.2.1 动机

使时钟系统具备环境感知能力, 温度与天气状况在板端和 PC 端同步呈现。

4.2.2 实现

weather_helper.py ——wttr.in 免 Key 接口:

```
1 def fetch_shanghai_weather(timeout=5):
2     data = requests.get(
3         "https://wttr.in/Shanghai?format=j1",
4         timeout=timeout).json()
5     current = data["current_condition"][0]
6     temp = int(current["temp_C"])
7     description = current["weatherDesc"][0]["value"]
8     return temp, map_condition(description), description
```

表 5: 天气缩写映射

wttr.in 原文	缩写	含义
Sunny, Clear	SUN	晴
Partly cloudy, Cloudy	CLD	多云
Overcast	OVC	阴天
Rain, Light rain, Heavy rain	RAI	雨
Snow, Light snow	SNO	雪
Fog, Mist	FOG	雾

调度策略：启动后 1 秒首次拉取，此后每 30 分钟 QTimer 自动刷新。

MCU 联动：PC 下发 *SET:WEA <T> <COND>，MCU 解析后：

- LED5 亮 = 晴；LED6 慢闪 = 雨/雪；LED7 亮 = 温度 ≥ 30℃
- 按 USER2 短显天气 5 秒，短显期间 *EVT:DISP 正常上报，PC 镜像自动跟随

4.3 E3：自动昼夜模式

4.3.1 实现

astral_helper.py ——按上海经纬度计算日出日落：

```
1 CITY = LocationInfo(  
2     "Shanghai", "China", "Asia/Shanghai", 31.2304, 121.4737)  
3  
4 def current_daynight():  
5     times = sun(CITY.observer, date=dt.date.today(),  
6               tzinfo=CITY.timezone)  
7     now = dt.datetime.now(times["sunrise"].tzinfo)  
8     mode = "DAY" if times["sunrise"] <= now <= times["sunset"] \  
9           else "NIGHT"  
10    return mode, times["sunrise"], times["sunset"]
```

PC 每分钟通过 QTimer 检查一次，跨越日出/日落时自动切换：

```
1 def auto_daynight(self):  
2     if not self.auto_mode_check.isChecked():  
3         return  
4     mode, sunrise, sunset = current_daynight()  
5     self.sun_label.setText(  
6         f"日升/日落： {sunrise:%H:%M} / {sunset:%H:%M}")
```

```
7 self.send_command(f"*SET:MODE {mode}")
```

表 6: 昼夜模式行为差异

模式	7SEG	LED	蜂鸣器
DAY	完整 8 位	全启用	正常
NIGHT	仅时/分 4 位	仅心跳	抑制

4.4 E4: 数据可视化看板

4.4.1 数据持久化

log_store.py——events.csv 追加记录:

```
1 class EventStore:
2     HEADER = ("timestamp", "type", "data")
3
4     def append(self, event_type: str, data: str):
5         with self.path.open("a", newline="",
6                               encoding="utf-8-sig") as file:
7             csv.writer(file).writerow([
8                 dt.datetime.now().
9                     strftime("%Y-%m-%d %H:%M:%S.%f")[:-3],
10                 event_type, data,
11             ])
```

4.4.2 三张并列图表 (chart_widget.py)

使用 matplotlib 的 FigureCanvasQTAgg 嵌入 Qt:

```
1 class ChartWidget(QWidget):
2     def update_from_rows(self, rows):
3         self.figure.clear()
4         axes = [self.figure.add_subplot(311),
5                 self.figure.add_subplot(312),
6                 self.figure.add_subplot(313)]
7
8         if not rows: # 数据为空友好提示
9             for ax in axes:
10                 ax.text(0.5, 0.5, "No data",
```

```
11             ha="center", va="center")
12         self.canvas.draw_idle()
13         return
14     # 图表 1: Alarm trigger time
15     ax.plot(xs, ys, marker="o",
16            linestyle="-", color="#ef4444")
17     # 图表 2: NTP delta (ms)
18     ax.bar(xs, ys, width=0.5, color="#22c55e")
19     # 图表 3: Key press heat
20     ax.barh(ypos, values, height=0.6, color="#3b82f6")
```

表 7: 数据看板图表一览

图表	类型	X 轴	Y 轴	数据源
闹钟触发分布	折线图	日期	小时（0-24）	ALARM 事件
NTP 误差	柱状图	日期	delta（ms）	SYNC 事件
按键热度	条形图	按键名	按压次数	KEY 事件

5 自主增加功能：专注倒计时系统

5.1 动机

传统闹钟基于绝对时间触发（如每天 07:00:00 响铃），适用于周期性唤醒场景。但在课堂演示、专注力训练、短时提醒等场景中，用户更需要基于**相对时长**的倒计时功能。该功能与基础闹钟形成互补：闹钟按绝对时间、倒计时按相对时间，二者独立并行。

5.2 设计

- **PC 侧**：在扩展面板「倒计时」分组框内提供一个 QSpinBox（范围 1-3599 秒，步长 1）和一个「开始倒计时」按钮，点击后下发 *SET:COUNTDOWN <seconds>。
- **MCU 侧**：接收 *SET:COUNTDOWN 后启动倒计时，期间数码管显示格式为 CDMSSS（如 CD0530 表示 5 分 30 秒），第 4 位小数点常亮作为标志。倒计时归零后，自动复用蜂鸣器驱动（节奏式响铃，最长 10 秒）并上报 *EVT:COUNTDOWN_DONE。
- **不新增依赖**：完全复用 MCU 既有的蜂鸣器、显示、LED 和串口事件上报机制。

5.3 实现

MCU 全局状态：

```

1 static bool g_countdown_active = false;
2 static uint32_t g_countdown_end_ms = 0;

```

MCU 协议入口：

```

1 if (Token_Matches(argv[0], "SET:COUNTDOWN")) {
2     uint16_t sec;
3     if (argc < 2 || !Parse_U16(argv[1], &sec) || sec == 0 || sec
        > 3599) {
4         UART_PutString("ERROR RANGE\r\n"); return;
5     }
6     g_countdown_end_ms = g_tick_ms + ((uint32_t)sec * 1000U);
7     g_countdown_active = true;
8     UART_PutString("OK\r\n");
9     return;
10 }

```

参数范围 1–3599 秒（约 1 小时内），超范围返回 ERROR RANGE；Parse_U16() 复用基础工具函数，拒绝负数与非数字输入。

MCU 显示集成：

```

1 // 倒计时进行中 格式 CD MM SS 第 4 位小数点作标志
2 if (g_countdown_active) {
3     uint32_t remain_ms = (g_countdown_end_ms > g_tick_ms) ? (
        g_countdown_end_ms - g_tick_ms) : 0;
4     uint32_t remain_s = (remain_ms + 999U) / 1000U; // 向上取整到
        秒
5     snprintf(text, sizeof(text), "CD%02lu%02lu ",
6             (unsigned long)(remain_s / 60U), (unsigned long)(
        remain_s % 60U));
7     dp = 0x08;
8     if (remain_s == 0) {
9         // 倒计时归零 转为蜂鸣器响铃 上报事件
10        g_countdown_active = false;
11        g_alarm.ringing = 1;
12        g_alarm_ring_start_ms = g_tick_ms;
13        g_ring_limit_ms = 10000;
14        UART_PutString("EVT:COUNTDOWN_DONE\r\n");
15    }
16 }

```

PC 端 UI:

```
1 focus_box = QGroupBox("倒计时")
2 row = QHBoxLayout(focus_box)
3 self.countdown_spin = self.spin(1, 3599, 300)
4 row.addWidget(self.countdown_spin)
5 row.addWidget(QLabel("秒"))
6 row.addStretch(1)
7 row.addWidget(self.button("开始倒计时", lambda: self.send_command(f
    "*SET:COUNTDOWN {self.countdown_spin.value()}"))))
```

5.4 演示

1. PC 扩展面板「倒计时」分组框中，将 QSpinBox 设为 10（秒）；
2. 点击「开始倒计时」；
3. S800 板数码管立即显示 CD0010，第 4 位小数点常亮，逐秒递减；
4. 10 秒倒计时结束时：
 - 蜂鸣器节奏式响铃（最长 10 秒自动停止）；
 - 数码管自动切回正常时钟显示；
 - PC 日志区出现 *EVT:COUNTDOWN_DONE；
 - 数据看板记录该事件。
5. **关键验证：**倒计时期间闹钟功能独立可用，互不干扰；串口心跳和 PC 镜像面板持续正常刷新。

6 难点解决与防崩溃机制

6.1 难点一：高频心跳包刷屏导致日志卡顿

现象：串口 1Hz 发送 *PONG、*EVT:DISP、*EVT:LED，QPlainTextEdit 每秒追加 3 行日志，运行 5 分钟后日志行数达到 900+，内存持续增长，滚动卡顿。

解决方案：

```
1 # main.py
2 self.log.setMaximumBlockCount(1000)    # 最多 1000 行，超出丢弃最早行
3
4 # protocol.py
5 def is_heartbeat(line: str) -> bool:
6     return line.startswith("*PONG") \
7         or line.startswith("*EVT:DISP") \
```

```
8         or line.startswith("*EVT:LED")
9
10 # main.py
11 if self.show_heartbeat.isChecked() or not is_heartbeat(line):
12     self.add_log(kind, f"<- {line}")
```

效果：「显示心跳」复选框（默认关闭）过滤后，日志区仅显示业务事件（闹钟、按键、ERROR），高频心跳包不再干扰。

6.2 难点二：网络超时导致 UI 假死

现象：NTP 同步或天气获取时，`requests.get()` 阻塞 5–10 秒，Qt 事件循环被卡住，窗口无法拖动、按钮无响应。

解决方案：引入 `NetworkWorker(QThread)`，将网络请求完全移到子线程：

```
1 class NetworkWorker(QThread):
2     done = pyqtSignal(str, object)
3     failed = pyqtSignal(str, str)
4
5     def run(self):
6         try:
7             if self.kind == "ntp":
8                 self.done.emit(self.kind, fetch_network_time())
9             elif self.kind == "weather":
10                 self.done.emit(self.kind, fetch_shanghai_weather())
11         except Exception as exc:
12             self.failed.emit(self.kind, str(exc))
```

效果：网络请求期间 GUI 完全流畅，3 台 NTP 服务器自动切换容错。

6.3 难点三：极端工程边界下的防崩溃设计

全局异常兜底：

```
1 def install_excepthook():
2     def hook(exc_type, exc, tb):
3         message = "".join(
4             traceback.format_exception(exc_type, exc, tb))
5         sys.stderr.write(message)
6     try:
```

```
7         QMessageBox.critical(None, "意外错误", str(exc))
8     except Exception:
9         pass
10    sys.excepthook = hook
```

表 8: 异常场景处理表

#	场景	处理方式	位置
1	串口被占用	弹窗 + 控制关闭	serial_worker.py
2	写入失败	break 退出线程 + 连接状态置离线	serial_worker.py
3	MCU 回应 ERROR	红色日志 + 状态栏显示	main.py
4	协议解析异常	[PARSE ERR] 红色日志	main.py
5	MSG 含非 ASCII	弹窗警告, 不发送	main.py
6	网络超时	弹窗 + 红色日志	main.py
7	心跳超时 3s	指示灯变灰 + 离线状态	main.py
8	未连接时发指令	红色日志 "not connected"	main.py

效果：上述所有异常场景均被覆盖，GUI 永不闪退。演示时可故意拔 USB、断网、发送错误指令，系统报错而非崩溃。

7 相关文件

表 9: 源代码文件索引

组件	路径
MCU 全部自编代码	mcu/src/main.c (~2000 行)
PC 端主程序	pc_host/main.py (~820 行)
串口后台线程	pc_host/serial_worker.py (~85 行)
协议解析	pc_host/protocol.py (~31 行)
数字孪生面板	pc_host/twin_panel.py (~210 行)
NTP 对时	pc_host/ntp_helper.py (~20 行)
天气获取	pc_host/weather_helper.py (~43 行)
昼夜模式	pc_host/astral_helper.py (~15 行)
数据看板	pc_host/chart_widget.py (~118 行)
日志存储	pc_host/log_store.py (~30 行)
UI 界面布局	pc_host/ui_main_window.py
UI 源文件	pc_host/ui/main_window.ui

——文档结束——